

Análisis Línea por Línea - Árbol Binario

Clase Nodo

```
1 class Nodo:
2     def __init__(self, valor):
3         self.valor = valor           # 1. Almacena el valor del nodo (A, B, C, etc.)
4         self.izquierdo = None       # 2. Inicializa referencia al hijo izquierdo como vac a
5         self.derecho = None         # 3. Inicializa referencia al hijo derecho como vac a
```

Explicación:

- **Línea 1:** Constructor que recibe el valor del nodo
- **Línea 2:** Guarda el valor en el atributo `valor`
- **Línea 3:** Inicializa el hijo izquierdo como `None` (sin hijo)
- **Línea 4:** Inicializa el hijo derecho como `None` (sin hijo)

Clase ArbolBinario

```
1 class ArbolBinario:
2     def __init__(self):
3         self.raiz = None           # 1. Inicializa el rbol sin ra z (vac o)
```

Explicación:

- **Línea 1:** Constructor de la clase árbol
- **Línea 2:** Inicializa la raíz del árbol como `None` (árbol vacío)

Método insertar_lado

```
1 def insertar_lado(self, padre, valor, direccion):
2     nuevo_nodo = Nodo(valor)       # 1. Crea un nuevo nodo con el valor dado
3
4     if self.raiz is None:          # 2. Verifica si el rbol est vac o
5         self.raiz = nuevo_nodo     # 3. Si est vac o, el nuevo nodo es la ra z
6         return True                # 4. Retorna xito
7
8     nodo_padre = self._buscar_nodo(self.raiz, padre) # 5. Busca el nodo padre
9
10    if nodo_padre is None:          # 6. Si no encuentra el padre
11        print(f"Error: No se encontr el nodo padre con valor {padre}")
12        return False               # 7. Retorna error
13
14    if direccion == 'izquierdo':    # 8. Si la direcci n es izquierda
15        if nodo_padre.izquierdo is None: # 9. Verifica si no tiene hijo izquierdo
16            nodo_padre.izquierdo = nuevo_nodo # 10. Asigna el nuevo nodo como hijo izquierdo
17            return True             # 11. Retorna xito
18        else:
19            print(f"Error: El nodo {padre} ya tiene un hijo izquierdo")
20            return False            # 12. Retorna error si ya tiene hijo
21    elif direccion == 'derecho':    # 13. Si la direcci n es derecha
22        if nodo_padre.derecho is None: # 14. Verifica si no tiene hijo derecho
23            nodo_padre.derecho = nuevo_nodo # 15. Asigna el nuevo nodo como hijo derecho
24            return True             # 16. Retorna xito
25        else:
26            print(f"Error: El nodo {padre} ya tiene un hijo derecho")
27            return False            # 17. Retorna error si ya tiene hijo
28    else:
29        print("Error: Direcci n debe ser 'izquierdo' o 'derecho'")
30        return False               # 18. Retorna error si direcci n no v lida
```

Método de Búsqueda Recursiva

```
1 def _buscar_nodo(self, nodo_actual, valor):
2     if nodo_actual is None: # 1. Caso base: si el nodo es None
3         return None # 2. Retorna None (no encontrado)
4
5     if nodo_actual.valor == valor: # 3. Si encuentra el nodo con el valor buscado
6         return nodo_actual # 4. Retorna el nodo encontrado
7
8     resultado_izq = self._buscar_nodo(nodo_actual.izquierdo, valor) # 5. Busca en hijo izquierdo
9     if resultado_izq: # 6. Si lo encontré en el sub árbol izquierdo
10         return resultado_izq # 7. Retorna el resultado
11
12     return self._buscar_nodo(nodo_actual.derecho, valor) # 8. Busca en hijo derecho
```

Métodos de Visualización

```
1 def mostrar_arbol(self):
2     if not self.raiz: # 1. Verifica si el árbol está vacío
3         print("Árbol vacío") # 2. Mensaje de árbol vacío
4         return # 3. Termina la función
5
6     self._mostrar_recursivo(self.raiz, 0, "") # 4. Llama a función recursiva
7
8 def _mostrar_recursivo(self, nodo, nivel, prefijo):
9     if nodo is None: # 1. Caso base: si el nodo es None
10         return # 2. Termina la recursión
11
12     print(" " * nivel + prefijo + str(nodo.valor)) # 3. Imprime el nodo actual
13
14     if nodo.izquierdo or nodo.derecho: # 4. Si el nodo tiene hijos
15         if nodo.izquierdo: # 5. Si tiene hijo izquierdo
16             self._mostrar_recursivo(nodo.izquierdo, nivel + 1, " " + prefijo) # 6. Muestra hijo
17             izquierdo
18         if nodo.derecho: # 7. Si tiene hijo derecho
19             self._mostrar_recursivo(nodo.derecho, nivel + 1, " " + prefijo) # 8. Muestra hijo derecho
```

Función para Construir el Árbol del Ejercicio

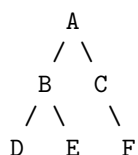
```
1 def construir_arbol_ejercicio():
2     arbol = ArbolBinario() # 1. Crea una instancia de árbol vacío
3
4     # Construye la estructura: A (B(izq) y C(der)), B (D(izq) y E(der)), C (F(izq))
5     arbol.insertar_lado(None, 'A', 'izquierdo') # 2. Inserta raíz A
6     arbol.insertar_lado('A', 'B', 'izquierdo') # 3. B como hijo izquierdo de A
7     arbol.insertar_lado('A', 'C', 'derecho') # 4. C como hijo derecho de A
8     arbol.insertar_lado('B', 'D', 'izquierdo') # 5. D como hijo izquierdo de B
9     arbol.insertar_lado('B', 'E', 'derecho') # 6. E como hijo derecho de B
10    arbol.insertar_lado('C', 'F', 'izquierdo') # 7. F como hijo izquierdo de C
11
12    return arbol # 8. Retorna el árbol construido
```

Ejecución Principal

```
1 # Ejecutar ejercicio 1
2 arbol = construir_arbol_ejercicio() # 1. Construye el árbol
3 print("Árbol construido:") # 2. Imprime título
4 arbol.mostrar_arbol() # 3. Muestra la estructura del árbol
```

Estructura Resultante

El código construye el siguiente árbol:



Salida esperada:

Árbol construido:

```
A
  B
    D
    E
  C
    F
```